Agent Laboratory

Un simulatore interattivo che modella un ecosistema di ricerca sul federated learning attraverso tre laboratori virtuali rappresentati in grafica pixel art 2D. I personaggi si muovono e interagiscono autonomamente, creando un ambiente di ricerca dinamico.

Stack Tecnologico

Frontend

- **Framework**: React + TypeScript
- **Rendering 2D**: Phaser 3 (sostituisce PixiJS)
- **UI Components**: Componenti custom in stile pixel art
- **Comunicazione**: Axios per API, Socket.io per WebSocket

Backend

- **Framework**: Python + FastAPI
- **Simulazione Multi-Agente**: Mesa
- **Federated Learning**: TensorFlow
- **Comunicazione**: WebSocket per comunicazione real-time

Struttura del Progetto

Il progetto è organizzato in due componenti principali:

Frontend (`/frontend`)

- Rendering grafico dei laboratori in Phaser 3
- Interfaccia utente in React
- Visualizzazione dello stato della simulazione

Backend (`/backend`)

- Logica di simulazione multi-agente con Mesa
- Implementazione degli algoritmi di federated learning
- API REST e WebSocket per comunicazione col frontend

Installazione e Setup

Prerequisiti

- Node.js (v14 o superiore)

- Python 3.9+
- npm o yarn

Setup Manuale (Senza Docker)

Backend

1. Navigare nella directory backend

   bash Copia

   ```bash
   cd agent-laboratory-v2/backend
   ```

2. Creare un ambiente virtuale Python (opzionale ma consigliato)

   bash Copia

   ```bash
   python -m venv venv
   source venv/bin/activate  # Linux/Mac
   venv\Scripts\activate     # Windows
   ```

3. Installare le dipendenze

   bash Copia

   ```bash
   pip install -r requirements.txt
   ```

4. Avviare il server backend

   bash Copia

   ```bash
   uvicorn api.main:app --host 0.0.0.0 --port 8000 --reload
   ```

Frontend

1. Navigare nella directory frontend

   bash Copia

   ```bash
   cd agent-laboratory-v2/frontend
   ```

2. Installare le dipendenze

   bash Copia

   ```bash
   npm install
   ```

3. Avviare il server di sviluppo

bash

Copia

```bash
npm start
```

Setup con Docker (Opzionale)

1. Assicurarsi che Docker e Docker Compose siano installati
2. Eseguire dalla directory principale:

bash

Copia

```bash
docker-compose up
```

Questo avvierà sia il frontend che il backend in container separati.

Utilizzo

- Accedere all'applicazione su http://localhost:3000
- Utilizzare i controlli per avviare, fermare o configurare la simulazione
- Esplorare i tre diversi laboratori e osservare le interazioni tra i ricercatori

Sviluppo

Convenzioni di Codice

- **Frontend**: Seguire le convenzioni TypeScript e React
- **Backend**: Seguire PEP 8 per Python
- Modularizzare il codice per mantenere file di dimensioni contenute
- Documentare le funzioni e le classi con docstring

Workflow di Sviluppo

1. Sviluppare e testare localmente
2. Versione le modifiche con Git
3. Eseguire i test automatici (quando implementati)
4. Creare pull request per review del codice

Licenza

MIT